Լաբորատոր աշխատանք N11

1. Աշխատանքի նպատակը

Տարբերակել և գործնականում կիրառել C++ լեզվում օբյեկտների փոխանցման երեք հիմնական եղանակները՝ **արժեքով** (by value), **հղումով** (by reference) և **հաստատուն հղումով** (const reference), ինչպես նաև յուրացնել պատճենող կոնստրուկտորի (copy constructor) և const անվտանգության սկզբունքները:

2. Աշխատանքի պահանջը (Տարբերակ №15 — Բջջային սակագնի օգտագործում)

- **Տվյալներ:** Ստեղծել MobileUsage դաս, որտեղ մասնավոր (private) բաժանորդների օգտագործած րոպեների զանգվածն է (std::vector<int>) և փաթեթում ներառված րոպեների քանակը (int p): Գերազանցող յուրաքանչյուր րոպեն արժե 20 դրամ:
- **Հաշվարկներ**:
  - Հաշվել լրացուցիչ վճարների ընդհանուր գումարը ($\sum \max(0, minutes - p) \cdot 20$).
  - Գտնել ամենամեծ լրացուցիչ վճարով բաժանորդի ինդեքսը:
  - Հաշվել սահմանաչափը գերազանցած բաժանորդների քանակը:
- **Ֆունկցիաներ**:
  - analyze(const MobileUsage&) – արտածում է վերը նշված երեք հաշվարկները՝ առանց օբյեկտը փոխելու:
  - changeCopy(MobileUsage object, int index, double value) – փոխում և արտածում է միայն տեղային պատճենը (օբյեկտը փոխանցվում է արժեքով):
  - reversed(const MobileUsage&) – վերադարձնում է նոր օբյեկտ՝ շրջված րոպեների զանգվածով և նույն p-ով:
- **Սահմանափակումներ:** Սկզբնական հավաքածուի չափը՝ $3 \le n \le 30$, բոլոր արժեքները ոչ բացասական են: Սխալ մուտքերի դեպքում վերադարձնել false և պահպանել նախկին տվյալները:

3. Տեսական մաս

1. **Արժեքով փոխանցում (Pass by Value):** Ստեղծվում է պարամետրի լիարժեք պատճեն (աշխատում է պատճենող կոնստրուկտորը): Ֆունկցիայի ներսում կատարված ցանկացած փոփոխություն **չի** ազդում սկզբնական բնօրինակ օբյեկտի վրա:

2. **Հղումով փոխանցում (Pass by Reference):** Ֆունկցիային է փոխանցվում բնօրինակ օբյեկտի հիշողության հասցեն: Ցանկացած փոփոխություն ուղղակիորեն փոխում է սկզբնական օբյեկտը:
3. **Հաստատուն հղումով փոխանցում (Pass by Const Reference):** Կիրառվում է օբյեկտն առանց պատճենման և առանց փոփոխելու իրավունքի (կարդալու նպատակով) արագ փոխանցելու համար: Ապահովում է բարձր արդյունավետություն և անվտանգություն (const երաշխավորում է, որ տվյալները չեն աղավաղվի):
4. **Private տվյալների հասանելիություն:** Արտաքին սովորական ֆունկցիաները դասի մասնավոր (private) դաշտերին ուղղակիորեն հասանելիություն չունեն և դրանց հետ աշխատում են հանրային (public) մեթոդների միջոցով (առանց friend հայտարարության):

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>

class MobileUsage {
private:
    std::vector<int> minutes;
    int p;

public:
    // Պարամետրերով կոնստրուկտոր՝ վավերացմամբ
    MobileUsage(const std::vector<int>& mins, int package_p) {
        if (mins.size() < 3 || mins.size() > 30 || package_p < 0) {
            std::cout << "Սխալ մուտքային տվյալներ (n-ը պետք է լինի [3, 30] միջակայքում, p
>= 0):\n";
            p = 0;
        } else {
            for (int m : mins) {
                if (m < 0) {
                    std::cout << "Րոպեները չեն կարող բացասական լինել:\n";
                    p = 0;
                    return;
                }
            }
            minutes = mins;
            p = package_p;
        }
    }

    // Պատճենող կոնստրուկտոր
```

```
    MobileUsage(const MobileUsage& other) : minutes(other.minutes), p(other.p) {
        // Ցուցադրման համար, թե երբ է կանչվում պատճենող կոնստրուկտորը
        // std::cout << "[Կանչվեց պատճենող կոնստրուկտորը]\n";
    }

    // Տպելու մեթոդ
    void print() const {
        std::cout << "Րոպեներ: [ ";
        for (size_t i = 0; i < minutes.size(); ++i) {
            std::cout << minutes[i] << (i + 1 < minutes.size() ? ", " : " ");
        }
        std::cout << "], Փաթեթ (p) = " << p << "\n";
    }

    // Set մեթոդ՝ արժեքի փոփոխության համար
    bool set(int index, double value) {
        int val = static_cast<int>(value);
        if (index < 0 || index >= static_cast<int>(minutes.size()) || val < 0) {
            return false;
        }
        minutes[index] = val;
        return true;
    }

    // Հանրային մեթոդներ տվյալների ստացման համար (private դաշտերի հետ աշխատելու համար)
    size_t size() const {
        return minutes.size();
    }

    int getMinute(int index) const {
        if (index < 0 || index >= static_cast<int>(minutes.size())) return 0;
        return minutes[index];
    }

    int getP() const {
        return p;
    }

    // Զանգվածը շրջելու մեթոդ
    void reverse() {
        std::reverse(minutes.begin(), minutes.end());
    }
};
```

```
// 1. analyze ֆունկցիա (const հղումով փոխանցում)
void analyze(const MobileUsage& obj) {
    long long totalFee = 0;
    int maxFee = -1;
    int maxFeeIndex = -1;
    int exceededCount = 0;

    for (size_t i = 0; i < obj.size(); ++i) {
        int mins = obj.getMinute(i);
        int extra = std::max(0, mins - obj.getP());
        int fee = extra * 20;

        totalFee += fee;

        if (fee > maxFee) {
            maxFee = fee;
            maxFeeIndex = static_cast<int>(i);
        }

        if (mins > obj.getP()) {
            exceededCount++;
        }
    }

    std::cout << "=== ANALYZE (Վերլուծություն) ===\n";
    std::cout << "Լրացուցիչ վճարների ընդհանուր գումարը: " << totalFee << " դրամ\n";
    std::cout << "Ամենամեծ լրացուցիչ վճարով բաժանորդի ինդեքսը: " << maxFeeIndex << "
(վճարը՝ " << maxFee << " դրամ)\n";
    std::cout << "Սահմանաչափը գերազանցած բաժանորդների քանակը: " <<
exceededCount << "\n";
}

// 2. changeCopy ֆունկցիա (արժեքով փոխանցում՝ ստեղծվում է տեղային պատճեն)
void changeCopy(MobileUsage obj, int index, double value) {
    std::cout << "=== CHANGE COPY (Փոփոխություն տեղային պատճենում) ===\n";
    if (obj.set(index, value)) {
        std::cout << "Տեղային պատճենում արժեքը հաջողությամբ փոխվեց:\n";
    } else {
        std::cout << "Սխալ ինդեքս կամ արժեք changeCopy-ում:\n";
    }
    std::cout << "Տեղային պատճենի վիճակը փոփոխությունից հետո:\n";
    obj.print();
}
```

```
// 3. reversed ֆունկցիա (վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով)
MobileUsage reversed(const MobileUsage& obj) {
    MobileUsage result(obj); // Օգտագործվում է պատճենող կոնստրուկտորը
    result.reverse();
    return result;
}

int main() {
    // Փորձնական տվյալներ՝ minutes = {80, 120, 200, 50}, p = 100
    std::vector<int> initialMinutes = {80, 120, 200, 50};
    int initialP = 100;

    MobileUsage subscriberPortfolio(initialMinutes, initialP);

    std::cout << "--- Բնօրինակ օբյեկտի սկզբնական վիճակը ---\n";
    subscriberPortfolio.print();
    std::cout << "\n";

    // Analyze կանչ
    analyze(subscriberPortfolio);
    std::cout << "\n";

    // ChangeCopy կանչ (2-րդ ինդեքսի արժեքը փոխում ենք 250-ի)
    changeCopy(subscriberPortfolio, 2, 250);
    std::cout << "\n";

    // Ստուգում ենք, որ բնօրինակը չի փոխվել արժեքով փոխանցման պատճառով
    std::cout << "--- Բնօրինակ օբյեկտը changeCopy-ից հետո (պետք է անփոփոխ մնա) ---\n";
    subscriberPortfolio.print();
    std::cout << "\n";

    // Reversed կանչ
    MobileUsage revPortfolio = reversed(subscriberPortfolio);
    std::cout << "--- Շրջված օբյեկտը (reversed) ---\n";
    revPortfolio.print();
    std::cout << "\n";

    std::cout << "--- Բնօրինակը շրջված օբյեկտը ստանալուց հետո (անփոփոխ է) ---\n";
    subscriberPortfolio.print();

    return 0;
}
```